

Uninformed Search on Deterministic Domains: BFS

Returning a Plan



```
plan BreadthFirstSearch () { //operates over a (deterministic) domain
    Queue q;
    q.enqueue( (s0,empty) );
    while (!q.empty()) {
        StatePlan (s,p) = q.dequeue();
        if (!marked(s)) {
            mark(s);
            if (goal(s)) return p;
            forall (a in alpha(s))
                q.enqueue( (delta(s,a), p+a) );
        }
    }
    return noplan;
}
```

Uninformed Search on Deterministic Domains: DFS

Returning a Plan



```
plan DepthFirstSearch () { //operates over a (deterministic) domain
    Stack t;
    t.push( (s0,empty) );
    while (!t.empty()) {
        StatePlan (s,p) = t.pop();
        if (!marked(s)) {
            mark(s);
            if (goal(s)) return p;
            forall (a in alpha(s))
                t.push( (delta(s,a), p+a) );
        }
    }
    return noplan;
}
```

Exercise: Use BFS and DFS to Return a Plan

Axiomatize the following scenario, appropriately with action precondition and effect axioms, and obtain successor state axioms.

Fluents:

- **boxClosed(s)** - The box is closed in situation s.
- **robotCloseToBox(s)** - The robot is close to the box in situation s.
- **objectInsideBox(s)** - The object is inside the box in situation s.

*Task: extract the object from the box
i.e.
Goal: not objectInsideBox*

Actions:

- **moveCloseToBox** - The robot moves close to the box. This can be done if the robot is not close to the box, and has the effect that the robot will be close to the box.
- **openBox** - The robot opens the box. This can be done if the robot is close to the box and the box is closed, and has the effect that the box will be open (that is, not closed).
- **extractObject** - The robot extracts the object inside the box. This requires that the box is open and the robot is close to the box, and has the effect that the object will be outside the box.

Initial situation description: Initially the robot is not close to the box, the box is closed, and the object is inside the box.

PDDL

```
(define (domain box)                                domain
(:requirements :strips :negative-preconditions)
(:predicates (boxClosed)
              (robotCloseToBox)
              (objInsideBox)
)

(:action moveCloseToBox
  :parameters ()
  :precondition (and (not (robotCloseToBox)))
  :effect (and (robotCloseToBox))
)

(:action openBox
  :parameters ()
  :precondition (and (robotCloseToBox) (boxClosed))
  :effect (and (not (boxClosed)))
)

(:action extractObject
  :parameters ()
  :precondition (and (not (boxClosed)) (robotCloseToBox))
  :effect (and (not (objInsideBox)))
))
```

```
(define (problem box-1)                               problem
(:domain box)
(:objects )

(:init
  ;; (not (robotCloseToBox))
  (boxClosed)
  (objInsideBox)
)

(:goal
  (not (objInsideBox))
))
```

Heuristic Search

Ramon Fraga Pereira

Sapienza Università di Roma, Italia

MSc in Engineering in Computer Science

May, 2022



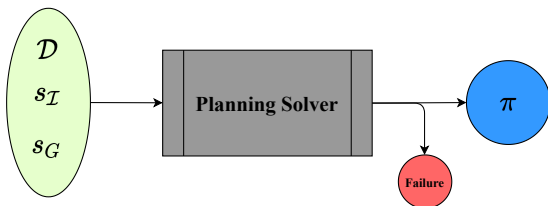
SAPIENZA
UNIVERSITÀ DI ROMA

Learn About:

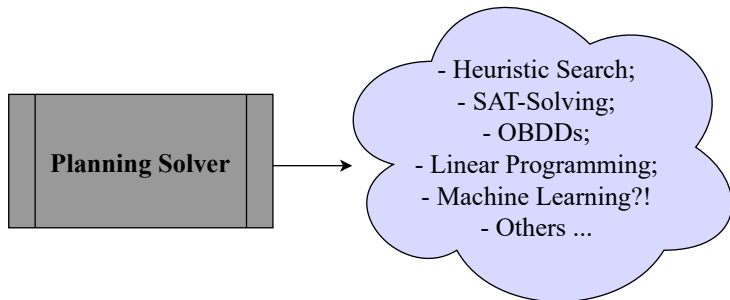
- Deterministic Heuristic Search: A^* ;
- Non-Deterministic Heuristic Search: AO^* ;

AI Planning

- In **AI Planning**, a *plan* π is synthesized from a *domain model* $\mathcal{D}$, an *initial state* s_I and a *goal (condition) state* s_G ;
- A *domain model* $\mathcal{D}$ defines the **properties** and **actions** of a particular environment.



- How to solve **AI Planning**?!



- In this lecture, we are going to discuss **Heuristic Search for Deterministic and Non-Deterministic Environments**.

AI Planning Applications¹

- Warehouse Logistics;
- Robotics;
- Agriculture;
- Underwater Operations;
- Space Operations;

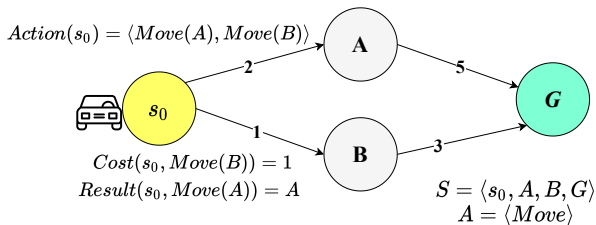


¹ <http://aiplan4eu-project.eu/>

Informed and Uniformed Search

State-Space Search Problems

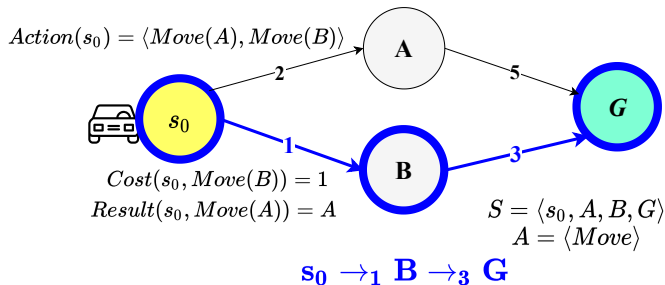
- *State-space search problems*² are among the “simplest” and most important classes of AI problems.
 - S is a set of **possible states**;
 - A is a set of **possible actions**;
 - $Action(s)$ defines that **applicable actions** in s ;
 - $Result(s, a)$ returns the **resulting state** when performing a in s ;
 - $Cost(s, a)$ is the **cost** of performing a in s .



² *State-space search problems* are **P-SPACE Complete**.

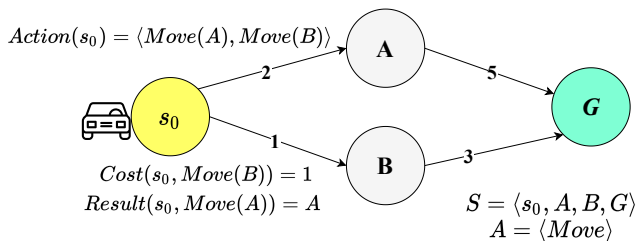
State-Space Search Problems

- **Solution:** A sequence of actions (a plan) that achieves a goal state G from an *initial state* s_0 ;
- **Performance measure:** minimize the total cost of *plan*.



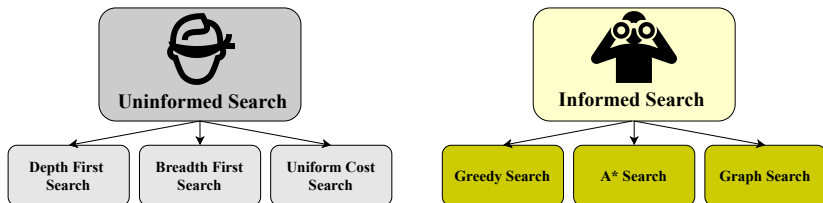
State-Space Search Notation

- States = Nodes (Vertices);
 - s_0 , A, B, G;
- Actions = Edges (Links);
 - $s_0 \rightarrow_2 A$;
 - $s_0 \rightarrow_1 B$;
 - $A \rightarrow_5 G$;
 - $B \rightarrow_3 A$;



Search Algorithms: Uninformed vs. Informed Search

- **Uninformed Search** (*Blind Search*) is a searching algorithm that uses no information about from the current state s to the goal G .
- **Informed Search** (*Heuristic Search*) is a searching algorithm that uses information about the estimated distance from the current state s to the goal G .



(Informed) Heuristic Search

- *Heuristic Search* is a type of algorithm in which the goal state is used to **guide the search** by means of **evaluation functions** $f(s)$;
- **Heuristic functions** $h(s)$ estimate the cost to achieve the goals state G from a state s ;

